

AdaBoost 分类算法介绍性报告

2312473-李一珩

2026-06-13

- 1. 引言与背景
- 2. 课本概念理解
 - 2.1 “Weak Learner”(弱分类器)
 - 2.2 “Iteratively”(迭代性)
 - 2.3 “Paying more attention / Giving more weight”
- 3. 数学原理与公式推导
 - Step 1: 初始权重分布
 - Step 2: 迭代 ($m = 1, 2, \dots, M$)
 - Step 3: 构建最终分类器
- 4. 算法逻辑
 - 4.1 AdaBoost 核心函数
- 5. 编程实现: 利用adaboost进行Titanic 生存预测
 - 5.1 数据准备与预处理
 - 5.2 模型训练
 - 5.3 误差率收敛
- 6. 总结
 - 6.1代码
 - 6.2特点

1. 引言与背景

在机器学习领域，**集成学习 (Ensemble Learning)** 是一种通过组合多个基分类器来完成学习任务的策略。根据基分类器生成方式的不同，Ensemble Learning可以分为两类：一种是基分类器之间存在强依赖关系、必须串行生成的 **Boosting**；另一种是基分类器之间不存在强依赖关系、可同时生成的 **Bagging**，典型代表是随机森林。

对于 **AdaBoost (Adaptive Boosting, 自适应提升算法)**，是由 Yoav Freund 和 Robert Schapire 于 1997 年提出的，证明了“**强可学习 (Strongly Learnable)**”与“**弱可学习 (Weakly Learnable)**”是等价的——即只要存在一个比随机猜测稍微好一点的弱分类器，我们就能通过某种机制将其提升为准确率极高的强分类器。

2. 课本概念理解

“The algorithm is based on the idea of generating weak learners that iteratively learn a larger portion of the difficult-to-classify examples... by paying more attention (that is, giving more weight) to often misclassified examples.”

2.1 “Weak Learner”(弱分类器)

弱分类器是指分类准确率仅比随机猜测（对于二分类而言，即准确率 $> 50\%$ ）略高的模型。在编程实现中，我们通常采用**决策树桩 (Decision Stump)**，即深度仅为 1 的单层决策树。它每次只利用一个特征的一个阈值进行切分，结构极其简单，计算速度快，非常适合作为 AdaBoost 的基础。

2.2 “Iteratively”(迭代性)

AdaBoost 不是一次性训练所有模型的。它需要经历 M 轮循环。在第 m 轮时，当前模型会利用前一轮 $m - 1$ 留下来的“经验”来进行训练。这种串行结构意味着后一个分类器是在前一个分类器的基础上进行“纠错”。

2.3 “Paying more attention / Giving more weight”

这是 AdaBoost 算法中适应性 (Adaptive) 的体现。在每一次迭代开始前，算法会为训练集中的每一个样本分配一个权重 (Weight)。

初始状态：所有样本的权重相等（即 $1/N$ ）。

自适应调整：如果某个样本在当前轮次被**正确分类**，那么在进入下一轮前，它的权重会被**缩小**；反之，如果它被**错误分类**，它的权重会被**放大**。

结果：随着迭代的进行，那些始终无法准确分类的样本会获得越来越大的权重。这就要求下一轮的弱分类器为了降低整体的加权误差率，必须将注意力 (Attention) 集中在这些无法准确分类的样本上。

3. 数学原理与公式推导

对于一个二分类训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ ，其中输入 $x_i \in \mathbb{R}^n$ ，输出类别标记 $y_i \in \{-1, +1\}$ 。AdaBoost 的步骤如下：

Step 1: 初始权重分布

在没有任何先验知识的情况下，假设数据集均匀分布，每个样本的初始权重相同：

$$D_1 = (w_{11}, w_{12}, \dots, w_{1N}), \quad w_{1i} = \frac{1}{N}, \quad i = 1, 2, \dots, N$$

Step 2: 迭代 ($m = 1, 2, \dots, M$)

对于每一轮迭代 m ：

(a) 训练当前基分类器 基于具有当前权值分布 D_m 的训练集，训练出弱分类器：

$$G_m(x) : \mathcal{X} \rightarrow \{-1, +1\}$$

(b) 计算分类误差率 e_m 计算 $G_m(x)$ 在带有权重的训练集上的分类误差率：

$$e_m = \sum_{i=1}^N w_{mi} I(G_m(x_i) \neq y_i)$$

公式含义：误差率 e_m 不是简单的错分样本个数比例，而是**被错分样本的权重之和**。

(c) 计算基分类器的权重系数 α_m 根据误差率，计算当前分类器在最终集成模型中的权重系数：

$$\alpha_m = \frac{1}{2} \ln \frac{1 - e_m}{e_m}$$

公式含义：这是一个关于 e_m 单调递减的函数。当误差率 $e_m < 0.5$ 时， $\alpha_m > 0$ ，且 e_m 越小， α_m 越大。这意味着分类准确率越高的弱分类器，在最终决策时权重系数越大。

(d) 更新样本权值分布 为下一轮迭代准备新的样本权重：

$$w_{m+1,i} = \frac{w_{mi}}{Z_m} \exp(-\alpha_m y_i G_m(x_i))$$

其中 Z_m 是规范化因子，确保所有权重之和为 1：

$$Z_m = \sum_{i=1}^N w_{mi} \exp(-\alpha_m y_i G_m(x_i))$$

重点： 注意到指数项 $-\alpha_m y_i G_m(x_i)$ 。如果预测正确， $y_i = G_m(x_i)$ ，乘积为 $+1$ ，指数项为 $-\alpha_m < 0$ ， w_{mi} 乘上一个小于 1 的数，**权重降低**。如果预测错误， $y_i \neq G_m(x_i)$ ，乘积为 -1 ，指数项为 $+\alpha_m > 0$ ， w_{mi} 乘上一个大于 1 的数，**权重放大**。

Step 3: 构建最终分类器

将所有弱分类器线性组合，并通过符号函数得到最终的强分类器：

$$G(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m G_m(x) \right)$$

4. 算法逻辑

不调用 `adaboost` 包，利用基础 `r` 包编写 `adaboost` 算法逻辑。

```
library(rpart) # 仅用于构建基分类器（树桩）
library(ggplot2) # 用于结果可视化
```

4.1 AdaBoost 核心函数

```

# 自定义 AdaBoost 训练函数
my_adaboost <- function(X, y, M = 50) {
  N <- length(y)

  # 步骤1: 初始化样本权重分布 D1
  w <- rep(1/N, N)

  models <- list()      # 存储每一轮的弱分类器
  alphas <- numeric(M)  # 存储每一轮分类器的话语权 alpha
  error_rates <- numeric(M) # 存储每一轮的加权误差率 e_m

  for (m in 1:M) {
    # 步骤2(a): 基于当前权重 w 训练弱分类器
    # 这里使用 rpart 构建深度为1的决策树 (Decision Stump)
    data_m <- cbind(X, y = y)
    fit <- rpart(y ~ ., data = data_m, weights = w,
                  method = "class",
                  control = rpart.control(maxdepth = 1, cp = -1, minsplit = 0))

    # 获取预测结果并转换为数值型
    pred_m <- as.numeric(as.character(predict(fit, X, type = "class")))

    # 步骤2(b): 计算加权分类误差率 e_m
    err_m <- sum(w[pred_m != y])

    # 边界情况处理: 如果误差率大于0.5, 说明连随机猜测都不如, 停止迭代
    if (err_m >= 0.5) {
      M <- m - 1
      break
    }

    # 如果误差率为0, 完美分类, 直接赋予极高权重并停止
    if (err_m == 0) {
      alphas[m] <- 5
      models[[m]] <- fit
      break
    }

    # 步骤2(c): 计算当前分类器的权重 alpha_m
    alpha_m <- 0.5 * log((1 - err_m) / err_m)

    # 保存本轮模型和参数
    alphas[m] <- alpha_m
    models[[m]] <- fit
    error_rates[m] <- err_m

    # 步骤2(d): 更新样本权重分布 w (被错分样本的权重将被指数放大)
  }
}

```

```
# 当  $pred\_m == y$  时,  $y * pred\_m = 1$ ; 当  $pred\_m != y$  时,  $y * pred\_m = -1$ 
w <- w * exp(-alpha_m * y * pred_m)

# 归一化因子  $Z_m$ , 使权重和为1
w <- w / sum(w)
}

# 裁剪多余的空位
alphas <- alphas[1:M]
error_rates <- error_rates[1:M]

list(models = models, alphas = alphas, errors = error_rates, M = M)
}

# 步骤3: 组合预测函数 (最终的强分类器)
predict_adaboost <- function(ada_model, newX) {
  M <- ada_model$M
  n_samples <- nrow(newX)
  f_x <- rep(0, n_samples)

  # 将所有弱分类器的结果按  $\alpha$  进行加权求和
  for (m in 1:M) {
    pred_m <- as.numeric(as.character(predict(ada_model$models[[m]], newX, type =
"class"))))
    f_x <- f_x + ada_model$alphas[m] * pred_m
  }

  # sign 函数: 大于0判断为正类(1), 小于0判断为负类(-1)
  sign(f_x)
}
```

5. 编程实现：利用adaboost进行Titanic 生存预测

为了检验上述代码的有效性，使用 R 语言自带的 Titanic 数据集。这是一个典型的二分类问题，特征包括舱位等级、性别、年龄段等。

5.1 数据准备与预处理

```
# Titanic 原始数据是一个高维数组/频数表，需要展开为标准数据框
titanic_df <- as.data.frame(Titanic)
titanic_data <- data.frame(
  Class = rep(titanic_df$Class, titanic_df$Freq),
  Sex = rep(titanic_df$Sex, titanic_df$Freq),
  Age = rep(titanic_df$Age, titanic_df$Freq),
  Survived = rep(titanic_df$Survived, titanic_df$Freq)
)

# 核心步骤：将分类标签映射为 {1, -1} 以满足 AdaBoost 的数学设定
titanic_data$y <- ifelse(titanic_data$Survived == "Yes", 1, -1)

# 随机划分训练集 (70%) 和测试集 (30%)
set.seed(2026)
n_total <- nrow(titanic_data)
train_idx <- sample(1:n_total, size = 0.7 * n_total)

train_data <- titanic_data[train_idx, ]
test_data <- titanic_data[-train_idx, ]

# 提取特征 X 和标签 y
X_train <- train_data[, c("Class", "Sex", "Age")]
y_train <- train_data$y

X_test <- test_data[, c("Class", "Sex", "Age")]
y_test <- test_data$y
```

5.2 模型训练

我们设定迭代次数 $M = 30$ ，观察集成后的模型表现。

```
# 运行我们手写的 AdaBoost 算法
M_iters <- 30
model_ada <- my_adaboost(X_train, y_train, M = M_iters)

# 在训练集上的表现
pred_train <- predict_adaboost(model_ada, X_train)
train_acc <- sum(pred_train == y_train) / length(y_train)

# 在未见过的测试集上的表现
pred_test <- predict_adaboost(model_ada, X_test)
test_acc <- sum(pred_test == y_test) / length(y_test)

cat(sprintf("经过 %d 轮迭代集成：\n", model_ada$M))
```

```
## 经过 30 轮迭代集成：
```

```
cat(sprintf("-> 训练集准确率: %.2f%%\n", train_acc * 100))
```

```
## -> 训练集准确率: 78.05%
```

```
cat(sprintf("-> 测试集准确率: %.2f%%\n", test_acc * 100))
```

```
## -> 测试集准确率: 77.31%
```

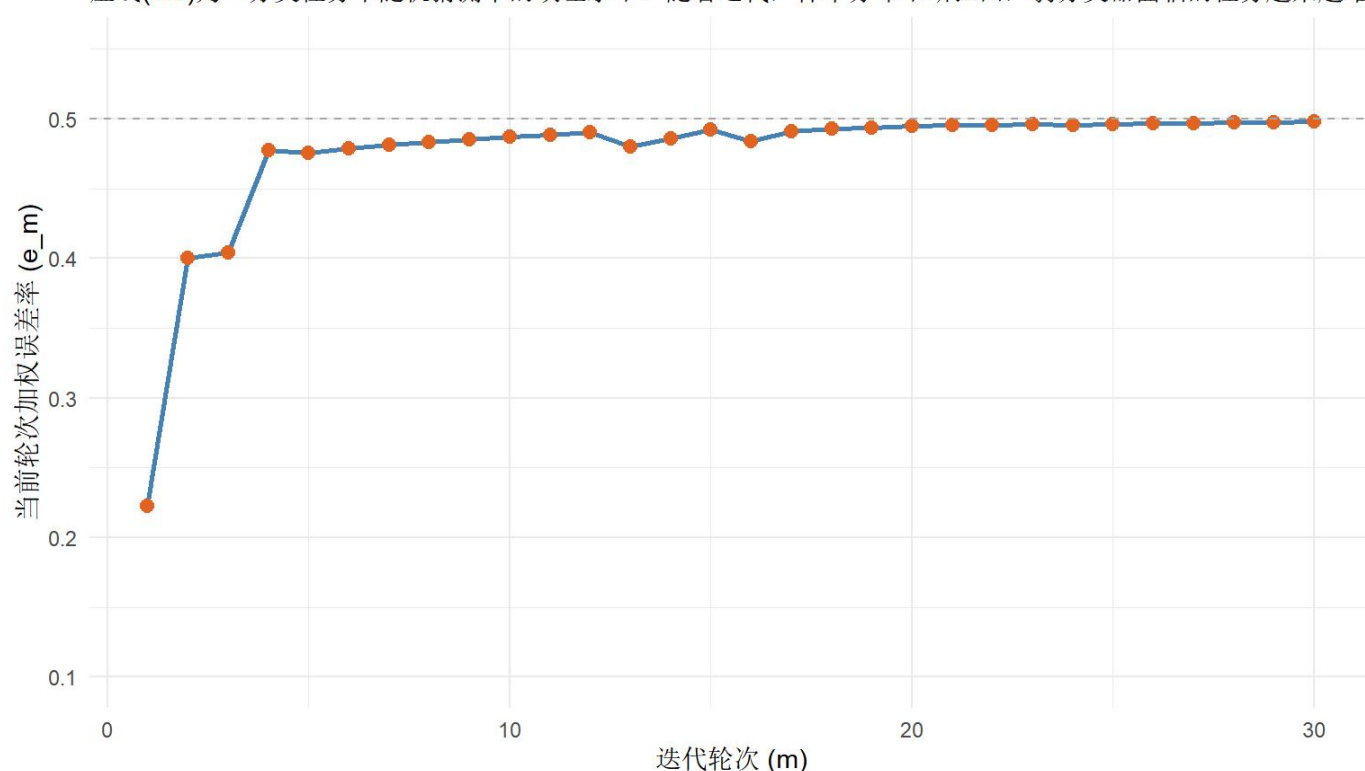
分析：尽管我们使用的仅仅是只能判断单个条件的“弱分类器”，仅通过船员的性别是否为女性来预测该船员最终是否存活，但通过 AdaBoost 机制集成 30 个这样的模型后，在测试集上的准确率依然达到了**77.31%**，证明了强分类器和弱分类器的一致性。

5.3 误差率收敛

为了直观展示 AdaBoost 是如何关注于那些难分样本，绘制出每一轮迭代中弱分类器的**加权分类误差率 e_m** 。

AdaBoost 训练过程：弱分类器加权误差率 (e_m) 演变

虚线(0.5)为二分类任务中随机猜测下的误差水平。随着迭代，样本分布不断扭曲，弱分类器面临的任务越来越难



图表解读：在图表中我们可以看到，通常第一轮的错误率是最低的（最简单的特征被找到了）。但随着算法不断放大之前错分样本的权重，后续生成的弱分类器面对样本分布含有大量的难分样本。因此，弱分类器自身的加权误差率 e_m 呈上升趋势，逐渐逼近 0.5（二分类任务中随机猜测的阈值）。这证明了算法在强制模型去学习难分样本（difficult-to-classify examples）。

6. 总结

6.1 代码

通过上述理论推导与代码实现：

1. **Iteratively learn** 对应了代码中 `for (m in 1:M)` 的外部循环框架。
2. **Weak learners** 对应了代码中每一次调用 `rpart` 时设定的 `maxdepth = 1` 参数。
3. **Paying more attention (giving more weight)** 对应了数学公式 $w \leftarrow w \cdot \exp(-\alpha \cdot y \cdot \hat{y})$ 的代码实现，通过调整下一轮损失函数的惩罚力度，引导模型不断解决分类边界上的难分样本。

6.2 特点

AdaBoost 算法展现了如何将无数个弱分类器，通过科学的权重更最终形成一个强分类器，算法共包括三个特点：

1. **动态调整样本权重**：在每一轮迭代后，算法会评估当前弱分类器的预测结果。被错误分类样本的权重将被增加，被正确分类样本的权重将被降低。这一机制使得后续生成的弱分类器在训练时，数据分布更倾向于未能正确分类的样本。**问题**：对数据中的异常值（Outliers）和噪声敏感。由于算法会放大被错分样本的权重，少量不可分的异常点会导致后续模型计算严重偏移，影响整体分类边界。
2. **基于误差率对分类器赋予权重**：在模型的集成环节，各个弱分类器的输出采用线性加权进行组合。每个弱分类器在总公式中的权重系数，由其在所属迭代轮次中的加权分类误差率决定。误差率越低的弱分类器，其被分配的权重数值越大。
3. **串行的训练结构**：AdaBoost 的基分类器按顺序逐个生成。由于第 m 个分类器的训练数据权重分布直接依赖于第 $m - 1$ 个分类器的输出结果，该算法的训练过程无法并行计算。**问题**：在部分情境下运算速度会受限制。